PRN:

Batch:

Experiment no 2

Title: Shared Memory Parallel Programming using OpenMP constructs and scheduling strategies.

Objective:

To understand and implement shared memory parallel programming using OpenMP constructs and analyze the impact of loop parallelization and scheduling strategies on program performance.

OpenMP concepts covered in this assignment:

- Parallel regions
- Thread creation
- Work sharing constructs
- Parallel loops
- Scheduling policies
- Performance analysis

Section A: OpenMP Basics

Task 1: Thread Creation and Identification using OpenMP

Write an OpenMP program of matrix-matrix addition by creating multiple threads and display thread information.

Requirements:

The program should:

1. Create a parallel region using:

#pragma omp parallel

2. Display:

- Thread ID
- Total number of threads

3. Execute the program with different thread counts:

- 2 threads

- 4 threads
- 8 threads

Use:

omp_set_num_threads()

Task 2: Parallel Loop Execution using OpenMP

Write an OpenMP program to compute the sum of N numbers.

Sequential version:

$$sum = \sum_{i=0}^{N-1} A\,[i]$$

Parallelize the loop using:

#pragma omp parallel for

Perform experiments for:

- N = 10,000
- N = 1,000,000
- N = 10,000,000

Record:

- Execution time
- Number of threads
- Speedup

Section B: Application Task

Task 3: Parallel Matrix Multiplication using OpenMP

Implement matrix multiplication using OpenMP.

Given:

$$C = A \times B$$

where:

- A = M × N matrix
- B = N × P matrix

- C = M × P matrix

Requirements:

Implement:

1. Sequential version

2. OpenMP parallel version

Parallelize the computation using:

#pragma omp parallel for

Perform experiments for matrix sizes:

- 500 × 500
- 1000 × 1000
- 2000 × 2000

Measure:

- Execution time
- Speedup
- Efficiency

Calculate:

$$Speedup = \frac{T_s}{T_p}$$

Task 4: Analyze OpenMP Scheduling Strategies

Modify the matrix multiplication program by applying different scheduling methods:

Static Scheduling

schedule(static)

Dynamic Scheduling

schedule(dynamic)

Guided Scheduling

schedule(guided)

Compare their performance.

Prepare a table:

Scheduling Type	Execution Time	Speedup
Static		
Dynamic		
Guided		

Section C

Task 5: Optimization of Parallel Matrix Multiplication using OpenMP

Optimize the matrix multiplication implementation using advanced OpenMP techniques.

Apply:

1. Loop collapsing

Use:

collapse(2)

or

collapse(3)

where applicable.

2. Thread affinity

Experiment with:

OMP_PROC_BIND=true

and

OMP_PLACES=cores

3. Compare different thread counts:

- 1 thread
- 2 threads
- 4 threads
- 8 threads
- Maximum available threads

Performance Analysis

Prepare a consolidated report:

Threads	Execution Time	Speedup	Efficiency
1			
2			
4			
8			

Review Questions

1. What is the difference between a process and a thread?
2. Explain the role of the following OpenMP directives:

   a) parallel
   b) for
   c) parallel for
3. Why does OpenMP require shared memory architecture?
4. Why does increasing the number of threads not always improve performance?
5. Compare static and dynamic scheduling.
6. Why can dynamic scheduling introduce additional overhead?
7. Explain why matrix multiplication performance depends on cache utilization.
8. Why does loop collapsing improve parallel performance?
9. Analyze why speedup decreases after a certain number of threads.
10. Explain the impact of thread affinity on HPC performance.

Submit in a doc file:

1. Source code:
   - Sequential implementation
   - OpenMP implementation
2. Execution outputs
3. Performance comparison table
4. Graphs:
   - Threads vs Execution Time
   - Threads vs Speedup
5. Answers to review questions